

How the spaceplane autopilot works

krpcLab · spaceplane · what the program does and what makes it change what it is doing. State of the code on 2026-10-10 (defaults fingerprint 101fc0e0).

The autopilot takes a winged vehicle (the target is the Mk3 shuttle `qs_shuttle2`) from orbit to a stop on the KSC runway. It does not follow a stored plan. Every tick it reads the vehicle's state, propagates the rest of the flight to the ground with an RK4 integrator (`trajectory.py`), and takes each decision from that fresh prediction. `autopilot.py` is the phase machine and the only code that talks to the game; `guidance.py` holds the control laws as pure functions; `environment.py` holds the aerodynamic table the vehicle measures for itself; every tunable is in `config.py`.

The phase machine

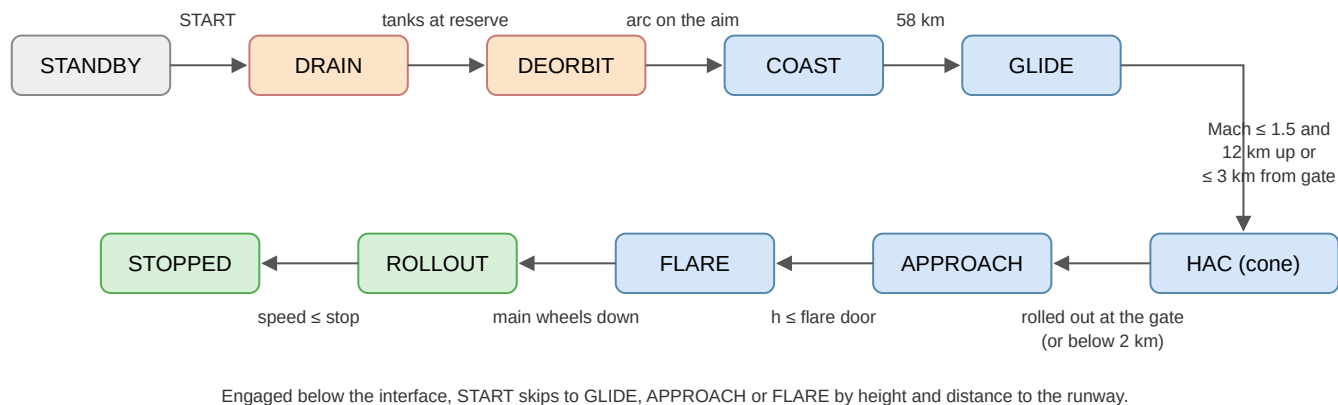


Figure 1. Phases and the conditions that advance them. Orange phases use the engine; blue are unpowered flight; green is on the ground.

Things that run in every phase

mechanism	what it does, and when
Prediction	A propagation to the ground of the law the vehicle will fly (the same function the controller calls), from the current state, each tick. The glide and the deorbit solve against it; the cone and approach plan from it.
Aerodynamic table	Swept in STANDBY with kRPC's <code>simulate_aerodynamic_force_at</code> over angle of attack and Mach; rows are re-probed in flight (a few per second). Stall speed, best L/D and approach speeds are read from it, not configured.
Attitude	kRPC's autopilot points the nose at the commanded (alpha, bank). Its response times are derived from the vessel's inertia and torque, retuned when surface authority grows with dynamic pressure. A roll damper lengthens roll response when bank swings grow and recovers it after.
RCS valve	Open when the pointing error exceeds 5°, closed 2 s after it is under 1.5°. Allowed in the burn, the coast, the glide above Mach 1 and during bank reversals.
Alpha ceiling	Learned: if the vehicle sits well below its commanded alpha the ceiling ratchets down to what it holds, and the prediction uses the same limit (<code>trajectory.Holdable</code> , binned by dynamic pressure). In the terminal glide the prediction also flies the alpha being achieved (see GLIDE).
Pitch trim	Above Mach 3 in the glide, the standing pitch input is moved off kRPC's integrators into a body-frame manual trim (<code>GLIDE_PITCH_OFFLOAD</code>). In the cone and approach the two nose canards are taken off kRPC and driven as a trim that walks the standing pitch input to zero (<code>CANARD_TRIM</code> , under 15° alpha, settled).
Brakes	Two built from the control surfaces, both probed in STANDBY: the split rudder (the two tail fins deployed in opposite directions: drag with no yaw or roll) and the spoiler (opposed flaps on the

	elevons, which are also the roll surfaces). Both slew at a limited rate. They are used only below the glide, in the order split rudder → spoiler → weave/S-turn (see HAC and APPROACH).
Time scale	The farm's time-scale plugin is set so each tick's game interval stays what the phase asked for; every log ends with a <code>loop rate</code> line.

Phase by phase

STANDBY — until START is pressed

Configures the craft (main gear brakes 200% / friction 10, nose gear brake off), measures wheel clearance and tail-strike angle, tunes the attitude controller, sweeps the aerodynamic table, and finds the brakes it can build from the surfaces (split rudder, opposed flaps). On START: below the 58 km interface → GLIDE / APPROACH / FLARE by height and distance; otherwise → DRAIN.

DRAIN — before the burn

Opens the craft's drain valves and empties propellant down to what the burn needs: the dv a deorbit can take ($200 \text{ m/s} \times 1.25$ margin) at the engine's vacuum Isp. Closes the valves, → DEORBIT. Doing it before the burn keeps every mass change upstream of the burn's own measurement.

DEORBIT — wait for a pass, then one burn

Each tick (2 s): choose the runway end; if the gate is 400–2300 km of arc ahead, search for a burn.

- **Search** (`deorbit_solution`): a grid then refinement over retrograde dv 10–400 m/s. For each candidate it propagates the two corners of what the glide may later command (`deorbit_window`): the shortest entry (high drag) and the longest (most lift), both flown with the measured alpha-tracking table (`Steer.tracked`). The burn chosen puts the gate an eighth of the window's width short of its centre (`DEORBIT_WINDOW_BIAS 0.25`), the side the glide can always correct by shortening.
- **No solution**: rails-warp to the next sample (the warp step is learned from how much orbit each warped tick covers) and ask again.
- **Line up**: once a burn is solved, kRPC's autopilot lets go and the game's own SAS takes the attitude: a stock maneuver node is placed 2 s ahead with the solved dv retrograde (moved every tick so it never passes), SAS in maneuver mode (retrograde if the mode is refused), RCS on. The burn is committed when the nose is within 3° of the burn vector on two ticks in a row, or after 120 s. At cutoff the node is removed, SAS off, and the autopilot re-engaged.
- **Burn**: hold the burn vector on SAS, limit thrust so the burn lasts at least 3 s, throttle tapered on the dv still owed. The stop test re-propagates the current arc each tick and measures the remaining dv against the chosen aim; when that propagation cannot answer (arrival past the 1500 s horizon), the burn stops on the solved dv and its exit line says *unmeasured*. The exit line also gives an open-loop dv estimate (throttle $\times$ the acceleration each interval began with, the limited one once the limiter is set) for reading against what was delivered. → COAST.

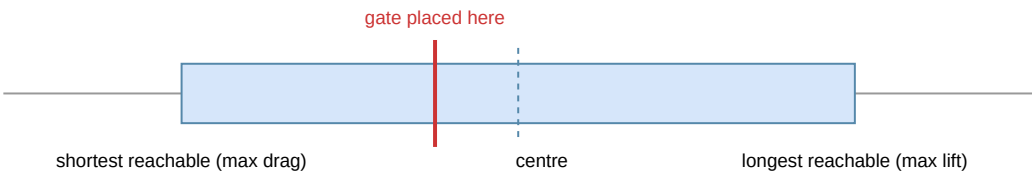


Figure 2. The deorbit window. The burn's dv moves the whole window; it is chosen so the gate sits 25% of the half-width short of centre.

COAST — to the 58 km interface

Swings to the entry attitude (22° alpha, 30° bank toward the runway) in vacuum, where holding it costs nothing, and rails-warps the part above the atmosphere (released 2 km above it). The bank's side is latched so the glide starts on the same lean. At 58 km → GLIDE.

GLIDE — the entry, Mach 25 to Mach 1.5

Each tick (1 s): solve alpha and bank against the prediction. What the solve nulls changes once, at Mach 4.5.

stretch	what the solve nulls
Hypersonic (above Mach 4.5)	Range. The along-track miss where the predicted arc crosses 12 km, aimed 500 m past the gate by a reserve that decays to nothing.
Terminal glide (Mach ≤ 4.5, latched)	The energy handed to the cone. The predicted energy height $h + v^2/2g$ where the arc meets the cone's entry test, against the energy the cone plans to be entered with: HAC_ALT_M (12 km) at the cone's target speed, plus a 1500 m margin (≈ 14.1 km). If the arc reaches 12 km before the entry circle, the path still to fly is priced at the cone's glide ratio and added; if that crossing is out of reach of the circle the miss is the <i>shorter</i> of the energy miss and the range shortfall, so a range short is never overridden by energy. If it crosses past the gate, the overshoot counts as surplus. The reserve is zero. The log's <code>long=</code> is then this energy surplus in metres, beside <code>tE=</code> (the target).
control	role
alpha (20–40°)	Downrange / energy: more alpha, more drag. Solved by sampling (both stops, best glide, the middle, a Newton step) against the miss, rate-limited to 3°/s, held still during a bank reversal, and clamped by speed (a cap holds the approach speed near the end) and by the learned ceiling. In the terminal glide the prediction flies <code>min(command, a 10 s running mean of the achieved alpha + 1°)</code> , and while the predicted energy is in surplus the solve may only <i>raise</i> alpha — commands above what the vehicle holds all predict alike, so lowering would ratchet the command and the vehicle down together.
bank magnitude (30–70°)	Takes what alpha leaves: rolling lift off vertical sinks the vehicle into denser air. When both are at their stops the solve returns the longest/shortest arc it can reach (<code>slv=max</code>).
bank sign	Cross-track: lean toward the runway; reverse when either the angle between the ground track and the bearing to the gate leaves a deadband of 0.012° per km to run (0.5°–12°), never narrower than the heading the vehicle turns through in one roll time-to-peak at its present bank ($g \cdot \tan bank /v \times$ the roll damper's measured time to peak, capped at 12°), or the predicted cross-track leaves a deadband of 20 m per km (500 m–40 km). A reversal commits once the commanded bank has passed 30° on the new side. Bank slews at a rate measured from the vehicle's roll.

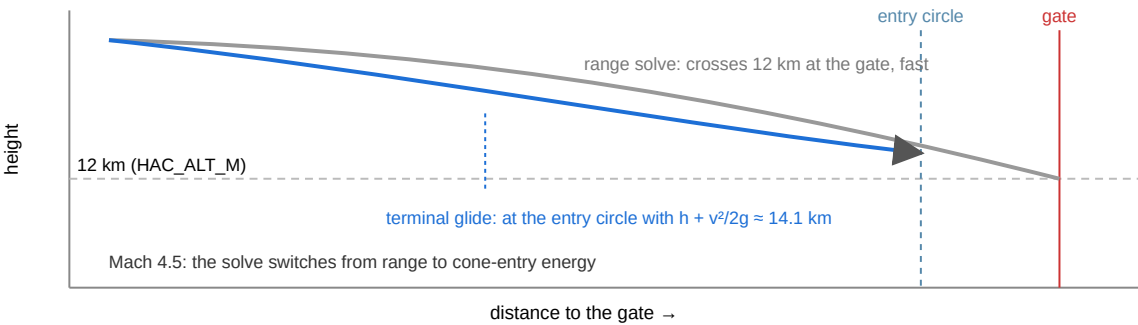


Figure 3. Below Mach 4.5 the glide aims at the energy the cone plans for, not at a range crossing that a steep, fast arc also satisfies.

At Mach ≤ 1.5 and either 12 km over the field or within 3 km of the gate: drop the terminal-glide target, release the alpha ceiling for the landing, pick the runway end and turn direction that are cheaper from here (`hac_choose`) → HAC.

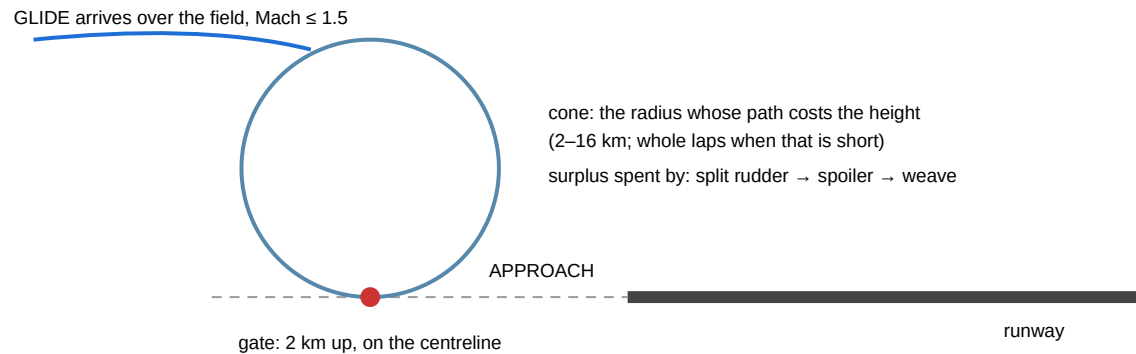


Figure 4. The heading-alignment cone. Whatever energy the entry delivers becomes a turn radius and brake; the rollout is the gate.

HAC — the heading-alignment cone

Each tick: re-plan the circle from the present state (`guidance.hac`).

- **Radius:** the circle (tangent to the extended centreline, ending at the gate) whose path, priced down a ladder of 500 m slices at the glide ratio the vehicle flies there, spends the height above the gate. The plan prices the path at the braked L/D, so the brake below is part of the plan rather than learned away. 2–16 km; a whole lap is added only when even the widest circle is short of path and the lap fits. The radius moves at a limited rate.
- **Speed and alpha:** hold the cone's target speed (from the measured stall) with alpha up to 22°.
- **Split rudder, first** (`HAC_SPLIT_BRAKE`): each tick it solves the L/D factor that spends the height to the gate over the planned path, $\text{path} / ((h - \text{gate}) \times \text{clean L/D})$, and opens to the angle that gives it (table `HAC_SPLIT_FACTOR`, 0–38°). It fades out over the 15% below the cone's target speed.
- **Spoiler, second** (`BRAKES_BEFORE_WEAVE`): only once the split rudder is at its stop, opened in proportion to the surplus height, full at the weave's threshold (800 m). It deploys the roll surfaces, so it is pulled in at once whenever the bank is off its command.
- **Bank and weave, last:** the bank that flies the circle, capped at 45°, plus a weave (up to 50° off track, starting on the side the vehicle is already banked) when more than 800 m of surplus height remains — and only once the split rudder is at its stop or faded out by low speed (`HAC_WEAVE_AFTER_BRAKE`).
- **Exit:** lined up (within 12°), within 900 m of the gate, no lap left, and no more than the approach can absorb above its need → `APPROACH`. Below the gate's 2 km → `APPROACH` regardless.

APPROACH — final, 2 km to the flare

Each tick (0.1 s): `guidance.approach` plus the brakes.

- **Aim point:** 1800 m past the threshold (the approach aims 1000 m nearer, so the flare's float lands in the zone).
- **Speed, not path:** alpha is set from the descent angle that holds the target speed (2.25× stall), from $\frac{dv}{dt} = g \sin\theta - D/m$, so it can make speed by steepening as well as lose it.
- **Split rudder** (`APPROACH_SPLIT_BRAKE`): opens to the L/D factor the geometry asks for, $\text{distance} / (\text{height} \times \text{best L/D})$, at 10°/s (38° cuts L/D by ~55%). Over 1.5× the design sink it fades out across the next 30% of that cap rather than stowing, and holds while the vehicle is faster than its target speed; under 1.45× stall it fades across the 15% below. It is in 4 s of sink above the flare door.
- **Spoiler:** once the split rudder is at its stop, opened in proportion to the excess height, full at the S-turn's threshold; stowed at the same point as the rudder, and pulled in whenever the bank is off its command.
- **Lateral:** a closure that brings cross-track to zero as the wheels arrive; S-turns (up to 45° of track) when more than 200 m of excess height remains, the aim is over 4 km away, and both brakes are at their stops.
- **Flare door:** 50 m + 2.5 s × sink rate. At or below it → `FLARE`.

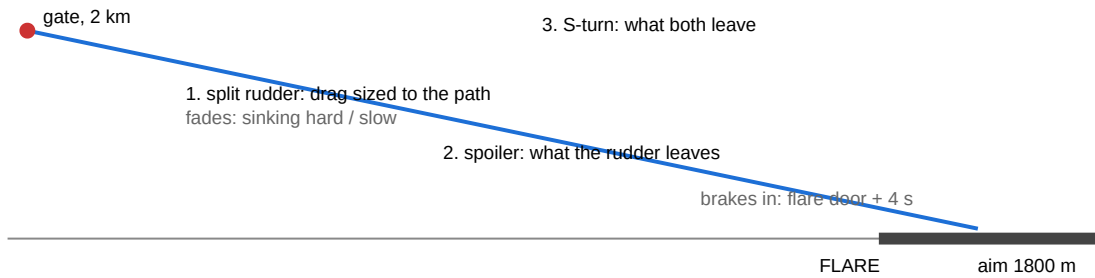


Figure 5. Final approach: the brakes buy path angle while alpha holds speed, in order; they are in before the flare.

FLARE — the last 50–100 m

Stows the brakes. Tracks a sink rate of $\sqrt{(v_{td}^2 + 2 a h)}$ that eases to the touchdown rate, with alpha capped by maximum lift (30°) and by the tail-strike angle. Above 60 m it may bank up to 12° (tapering to wings level) to close cross-track; below 30 m the nose is aligned with the runway, not the airflow. A proportional pitch term (0.08 per degree) helps kRPC hold the flare attitude. Gear down. Main wheels touching → ROLLOUT.

ROLLOUT — on the runway

Ground spoiler out on main-gear contact. The nose is ramped down from its touchdown attitude over 1.5 s (never stepped), the trims ramp out, full main-gear braking, nosewheel steering onto the centreline on cross-track plus its rate times a 3 s lead. At the stop speed → STOPPED, which logs the stop position and hands the vessel back.

Which law decides what

quantity	GLIDE	HAC	APPROACH	FLARE / ROLLOUT
downrange / energy	alpha, then bank magnitude — on range above Mach 4.5, on cone-entry energy below	circle radius and laps; then split rudder → spoiler → weave	split rudder → spoiler → S-turns	—
speed	speed cap on alpha (late)	alpha to the target speed	descent angle that holds speed	sink-rate law / wheel brakes
cross-track	bank sign, azimuth and cross-track deadbands	circle geometry	closure, wheels-time	bank $\leq 12^\circ$, then nosewheel
pitch trim	body-frame offload (Mach > 3)	canard trim	canard trim	P-term in the flare, ramped out

Logs

Nothing prints. Each run writes `logs/LOG<n>`: a `config` line (fields different from the defaults, plus the defaults fingerprint), one telemetry line per tick interval, every phase change and decision as an event (each brake's out/in with its reason; the terminal glide's start and its target), and a closing `loop rate` line. The `DOWN` line gives the stop along and across the runway and, from KSP's own latitude, how far north or south of the runway line it is (the space center lies to the south). `spaceplane/tools/` reads them (`landsum.py`, `armsum.py`, `conesum.py`, `oscsun.py`).